DBMS CAPSTONE PROJECT FINAL REPORT

Enterprise Automated ETL Data Pipeline & Interactive BI Analytics Platform

A Complete End-to-End Kimball Star Schema Data Warehouse, Automated Python ETL, and Interactive Analytics Solution

Student Name: Sami Siddiqui	**Course:** Database Management Systems (DBMS)
GitHub: github.com/SamiSiddiqui45	**Database:** Microsoft SQL Server (AnalyticsDW)
Repository: Enterprise-Automated-ETL-Data-Pipeline-BI-Analytics-Platform	**Submission Date:** November 2026

Executive Summary & Abstract

In modern retail and corporate organizations, raw transaction records are often generated in disparate flat-file CSV formats lacking referential integrity, consistent data types, and transactional constraints. This capstone project designs, implements, and verifies a production-ready, automated data engineering and business intelligence ecosystem using the canonical **Sample Superstore dataset (9,994 records, 21 columns)**. The platform features: (1) a multi-layered relational data warehouse on **Microsoft SQL Server Express (AnalyticsDW)** implementing staging (`stg`), dimensional warehouse (`dw`), and audit (`etl`) schemas following the **Kimball Star Schema** methodology; (2) an autonomous, modular Python 3.12 ETL framework enforcing 12 automated data-quality rules and quarantine audit mechanisms; and (3) an interactive business intelligence suite supporting real-time decision support in Streamlit and Microsoft Power BI Desktop. All metrics loaded into the data warehouse reconcile with **100% mathematical accuracy ($0.0000 variance)** against the raw source benchmarks ($2,297,200.86 Sales, $286,397.02 Profit across 5,009 distinct orders).

1. Problem Statement & Motivation

Organizations that store transaction records in flat CSV files face severe operational hurdles:

- **Data Quality Degradation:** Integer parsing truncates leading zeros on East Coast US postal codes (e.g. `01040` becomes `1040`), product titles vary over time for identical items, and invalid temporal dates emerge.
- **Lack of Relational Constraints:** Raw CSVs cannot enforce Primary Keys, Foreign Keys, or Unique constraints, risking duplicate sales records upon repeated ingestion.
- **Analytical Query Bottlenecks:** Executing multi-dimensional aggregations (e.g., Year-over-Year sales growth, regional customer segment analysis) directly on flat files causes memory exhaustion and excessive latency.
- **Silent Data Loss:** In traditional pipelines, invalid rows are often silently discarded without an audit trail, corrupting corporate balance sheets.

2. Project Objectives & System Requirements

To resolve these challenges, the platform was developed against six strict engineering objectives:
1. **Zero-Touch Pipeline Automation:** Build modular Python scripts (`extract`, `validate`, `transform`, `load`, `audit`) executed via a single orchestrator CLI (`python -m src.main`).
2. **Kimball Star Schema Architecture:** Structure the relational warehouse into an atomic line-item Fact table (`dw.FactSales`) surrounded by 5 conformed dimension tables with surrogate keys.
3. **Data Quality Quarantine Governance:** Implement 12 automated validation rules that redirect invalid rows to `etl.RejectedRecords` with diagnostic JSON payloads.
4. **Idempotency & Re-run Safety:** Ensure the pipeline can be re-run indefinitely without appending duplicate fact records or corrupting dimensions.
5. **Zero Variance Reconciliation:** Mathematically verify that loaded SQL Server totals exactly equal source benchmark totals down to the fourth decimal.
6. **Interactive Business Intelligence:** Provide executive stakeholders with high-impact visual analytics covering sales trends, customer behavior, and product discount destruction.

3. Relational Data Modeling & Star Schema Design

The core data warehouse resides in database **`AnalyticsDW`** hosted on **Microsoft SQL Server Express (`.\SQLEXPRESS`)**. The architecture segregates responsibilities across three logical schemas:

Schema	Table Name	Classification	Row Count	Key Architecture / Constraints
stg	stg.OrdersRaw	Staging Landing	19,988 (2 runs)	Preserves 100% raw source CSV fields with ETLRunID & UTC timestamp for auditab
dw	dw.DimDate	Calendar Dimension	2,922 rows	Continuous calendar dimension (2013-2020) supporting role-playing OrderDate & Sh
dw	dw.DimCustomer	Customer Dimension	793 rows	Surrogate CustomerKey, CustomerID (UQ), CustomerName, CustomerSegment.
dw	dw.DimProduct	Product Dimension	1,894 rows	Surrogate ProductKey, preserves all historical naming variants without data loss.
dw	dw.DimGeography	Geography Dimension	632 rows	Surrogate GeographyKey, normalized 5-digit zero-padded PostalCode (VARCHAR(5
dw	dw.DimShipMode	Fulfillment Dimension	4 rows	Surrogate ShipModeKey (Standard Class, Second Class, First Class, Same Day).
dw	dw.FactSales	Sales Fact Table	9,994 rows	Atomic line-item sales grain. 6 Foreign Keys, UQ on SourceRowID (idempotency enf
etl	etl.PipelineRuns	Audit Log Table	2 runs logged	Tracks batch execution status (SUCCESS/FAILED), start/end times, and row metrics
etl	etl.RejectedRecords	Quarantine Table	0 (100% Clean)	Quarantines invalid rows with error reason, offending column, and serialized raw JSO

4. Modular Python ETL Pipeline Implementation

The Python ETL application is organized into six maintainable modules in `src/`:
• **`extract.py`:** Safely reads the source CSV using Windows-1252 (`cp1252`) encoding, handles byte `0xA0` non-breaking spaces, and validates the presence of all 21 required column headers.
• **`validate.py`:** Executes 12 automated checks (mandatory business keys, numeric bounds on Sales >= 0, positive Quantity > 0, Discount between 0.0 and 1.0, and temporal sequence `ShipDate >= OrderDate`).
• **`transform.py`:** Normalizes postal codes to 5-digit zero-padded strings (`01040`), derives integer surrogate date keys (`YYYYMMDD`), and computes derived metrics (Estimated Cost: `Sales - Profit`, and safe Profit Margin).
• **`load.py`:** Resolves surrogate keys against dimension tables and batch-loads staging and fact records into SQL Server using `pyodbc.fast_executemany` with parameterized SQL. Employs `WHERE NOT EXISTS` to enforce idempotency.
• **`audit.py`:** Records execution telemetry in `etl.PipelineRuns` and manages quarantine logging in `etl.RejectedRecords`.
• **`main.py`:** CLI orchestrator coordinating extraction, validation, staging, dimension and fact loading, audit finalization, and post-load financial reconciliation.

5. Verification, Quality Assurance & Reconciliation Results

To establish academic and production credibility, the warehouse was subjected to automated financial reconciliation and unit test verification upon execution:

• **Automated Test Suite:** Pytest executed 12 out of 12 tests with a **100% pass rate in 0.73 seconds**, covering CSV readability, constraint enforcement, postal code normalization, date key logic, and SQL Server referential integrity.

• **Referential Integrity:** Execution of `sql/08_validation_queries.sql` confirmed **0 orphan foreign keys** across all six dimension relationships.

• **Financial Reconciliation:** Execution of `sql/10_reconciliation.sql` proved **zero variance ($0.0000)** between source flat data and the loaded warehouse:

Metric Name	Source CSV Benchmark	Loaded SQL FactSales	Variance	Reconciliation Status
Transaction Rows	9,994	9,994	0.0000	PASSED (100% Match)
Distinct Orders	5,009	5,009	0.0000	PASSED (100% Match)
Distinct Customers	793	793	0.0000	PASSED (100% Match)
Total Units Sold	37,873	37,873	0.0000	PASSED (100% Match)
Total Sales Revenue ($)	$2,297,200.8603	$2,297,200.8603	$0.0000	PASSED (Exact Match)
Total Net Profit ($)	$286,397.0217	$286,397.0217	$0.0000	PASSED (Exact Match)

6. Business Intelligence Analytics & Strategic Insights

The analytical layer features dual integration: an interactive live web platform (`src/dashboard.py`) powered by Streamlit and Plotly running on `http://localhost:8501`, and a Power BI Desktop dimensional model with pre-validated DAX measures. Key business discoveries include:

• **Technology Drives Profitability:** Technology generated $145,454.85 in profit on $836,154.03 sales, achieving the highest profit margin of **17.40%**. The Canon imageCLASS Copier proved the top individual earner ($25,199.98 net profit at 61.8% margin).

• **Discount Destruction in Furniture:** Furniture achieved $741,999.80 in sales but only $18,451.27 in profit (a meager 2.49% margin). The 'Tables' subcategory generated a net loss of **-$17,725.48** due to aggressive promotional discounts exceeding 26.1%.

• **Retail Seasonality:** Continuous 48-month analysis revealed steady surges during Q4 holiday shopping (September, November, December), peaking at $118,447.83 in November 2017.

• **Regional Dominance:** The West ($725.5K sales, $108.4K profit) and East ($678.8K sales, $91.5K profit) represent over 61% of total enterprise profitability.

7. Conclusion & Future Scope

This project demonstrates the complete lifecycle of modern data warehousing—from raw CSV ingestion and automated data-quality validation to Kimball Star Schema design and interactive executive visualization. The system achieves 100% idempotency, zero financial variance, and robust auditability. Future enhancements include cloud migration to Azure SQL Database, workflow DAG scheduling using Apache Airflow, and machine learning integration for customer churn and sales demand forecasting.